



Department of Computer Science

Lab Manual

of

MACHINE LEARNING

MCA521-4

Class Name: IV MCA

Master of Computer Application

2026

Prepared by:
Evana Joseph
2547225

Verified by:

Dr. Rupali Sunil Wagh Dr. Helen K Joy Dr. Shivangi Singh

Department Overview

Department of Computer Science of CHRIST (Deemed to be University) strives to shape outstanding computer professionals with ethical and human values to reshape nation's destiny. The training imparted aims to prepare young minds for the challenging opportunities in the IT industry with a global awareness rooted in the Indian soil, nourished and supported by experts in the field.

Vision

The Department of Computer Science endeavours to imbibe the vision of the University “**Excellence and Service**”. The department is committed to this philosophy which pervades every aspect and functioning of the department.

Mission

“To develop IT professionals with ethical and human values”. To accomplish our mission, the department encourages students to apply their acquired knowledge and skills towards professional achievements in their career. The department also moulds the students to be socially responsible and ethically sound.

Introduction to the Programme

Master of Computer Applications (MCA) is a post-graduate programme of CHRIST (Deemed to be University) and approved by the All India Council of Technical Education (AICTE). It is a two-year programme spread over six trimesters to bridge the chasm between computer studies and applications, bringing within reach of enthusiastic youngsters an excellent group of experienced and dedicated staff and experts, and an enviable infrastructure. MCA at CHRIST (Deemed to be University) strives to shape outstanding computer professionals with ethical and human values to reshape the nation's destiny. The training programme aims to prepare young minds for challenging opportunities in the IT industry with a global awareness rooted in the Indian soil, nourished and supported by experts in the field. The Department is committed to the motto “Excellence and Service,” and this philosophy pervades every aspect of its functioning.

Programme Objective

This programme is conceptualised and designed based on the strong commitment of the department to provide better quality education to the students. The principal objectives of this course are to:

- Heighten technological awareness
- Train future industry specialists
- Encourage effective software development
- Provide research training
- Involve students in innovative research
- Produce graduates with a two-year professional education in Computer Science with technical, professional, and communications skills.

Ethics and Human Values

1. Only proprietary or open-source software would be used for academic teaching and learning purposes.
2. Copying of programs from the internet, friends or from other sources is strictly discouraged since it impairs development of programming skills.
3. Unique Practical (Domain based) exercises ensure that the students don't get involved in code plagiarism.
4. Projects undertaken by students during the course are done in teams to improve collaborative work and synergy between team members.
5. Projects involve modularization which initiates students to take individual responsibility for common goals.
6. Passion for excellence is promoted among the students, be it in software development or project documentation.
7. Giving due credit to sources during the seminar and research assignment is promoted among the students
8. The course and its design enforce the practice of good referencing technique to improve the sense of integrity.
9. Courses involving group discussions and debates on ethical practices and human values are designed to sensitize the students in dealing with customers and members within the Organization.

Programme Outcomes:

- P01: Foundation Knowledge: Apply knowledge of mathematics, programming logic, and coding fundamentals for solution architecture and problem-solving.
- P02: Problem Analysis: Identify, review, formulate, and analyse problems primarily focussing on customer requirements using critical thinking frameworks.
- P03: Development of Solutions: Design, develop and investigate problems with as an innovative approach for solutions incorporating ESG/SDG goals.
- P04: Modern Tool Usage: Select, adapt, and apply modern computational tools such as the development of algorithms with an understanding of the limitations including human biases.
- P05: Individual and Teamwork: Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.
- P06: Project Management and Finance: Use the principles of project management such as scheduling, and work breakdown structure and be conversant with the principles of Finance for profitable project management.
- P07: Ethics: Commit to professional ethics in managing software projects with financial aspects.
- Learn to use new technologies for cyber security and insulate customers from malware
- P08: Life-long learning: Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

Programme Educational Objectives(PEO's)

- PEO1: Develop innovative and effective software solutions through research that addresses real-world challenges, grounded in a commitment to continuous learning and adaptability.
- PEO2: Advance the knowledge and practices in the field of computer applications through lifelong learning and rigorous research, supporting professional growth and academic excellence.
- PEO3: Exhibit ethical leadership, social responsibility to contribute and enhance community well-being by innovating solutions.

MCA521-4 – MACHINE LEARNING

Course Name: MACHINE LEARNING Code: MCA521-4

Total Hours: 75

L + P + T: 3 + 4 + 0

Max Marks: 100

Credits: 4

Course Type: Core Course

Course Category: Theo&Prac

Course Description

Machine Learning is a key area in computer applications that focuses on building models capable of making predictions or informed decisions based on data. A strong understanding of machine learning enables students to analyze and interpret complex datasets, develop predictive models, and extract meaningful insights. This course covers a wide range of machine learning concepts and techniques, including supervised and unsupervised learning. It provides a solid theoretical foundation while emphasizing practical, hands-on experience with algorithms and real-world data. The course equips students with the knowledge and skills required to apply machine learning techniques across various domains using modern tools and methodologies.

Course Objectives

This course enables students to understand the fundamental concepts of Machine Learning, including its core principles, terminology, and methodologies. Students will learn to differentiate between various types of learning algorithms such as supervised, unsupervised, and reinforcement learning, and recognize their appropriate applications.

Course Outcomes

After successful completion of the course, students will be able to:

- Explain the fundamental principles and scope of Machine Learning
- Implement modelling practices in machine learning for better generalisation
- Interpret predictive modelling results and performance of supervised machine learning techniques
- Assess the outcomes and effects of unsupervised machine learning processes
- Deploy robust machine learning models for interdisciplinary applications

Unit-1: INTRODUCTION TO MACHINE LEARNING Teaching Hours: 15 Hours

Introduction to AI – Knowledge Representation – Data – Representation of Data, Data Processing, Data Storage, Data Processing, Types of Data – Exploratory Data Analysis, Visualisations and Interpretation, Outliers

Machine Learning: Definition, Objectives, Components, Features, Applications – Traditional Programming v/s Machine Learning, Types of Machine Learning – Predictive Models, Statistical Inferences – Applications of Machine Learning

Challenges in ML: Insufficient Quantity of Data, Non-representative and Poor-Quality Data, Irrelevant Features, Estimation Estimation, Reducing Human Biases in Model Building, Ethical Use of Technology

Lab Exercises:

1. Data Preprocessing and visualisation
2. Data exploration and inferences

Unit-2: PROCESSES IN MACHINE LEARNING Teaching Hours: 15 Hours

Data Preparation and Model Selection: Effect of Data Preparation: Encoding, Nominal and Ordinal Representation, Feature Transformation and Feature Engineering, Generalisation, Normalisation, Sampling, Using Saved Weights, Model Selection Strategies: Overfitting, Underfitting, Bias-Variance Trade-off, Testing and Validation: Performance measures - Confusion matrix, Precision-Recall Trade off, F1 Score, ROC Curve, AUC, Error Analysis, Model Parameters, Hyperparameters, Hyperparameter Tuning, Cross Validation, Decision Functions: Introduction to Supervised Machine Learning Methods, Decision Functions, Decision Thresholds, Distance Measures, Hyperplanes, Regression & Classification Problems, Loss and Cost Functions, Linear Regression using OLS, Multi-class vs Multi-label, KNN Classification

Lab Exercises:

3. Saving weights of a generalised model.

4. Regression and Classification Evaluation Metrics: Illustration through Linear Regression and KNN Classification

Unit-3: SUPERVISED LEARNING ALGORITHMS

Teaching Hours: 15

Hours

Learning through Optimisation: Gradient Descent, Linear Regression, Logistic Regression
Computational Learning: Decision Trees, Naive-bayes classification, Support Vector Machines, Ensemble Learners: Learning, Voting, Bagging, Stacking, Boosting, Random Forests, GridSearchCV

Lab Exercises:

5. Regression through Gradient Descent

6. Comparison of Different Classifiers

7. Illustration of Ensemble Learning Methods

Unit-4: UNSUPERVISED LEARNING AND DIMENSIONALITY REDUCTION

Teaching

Hours: 15 Hours

Introduction: Types of Unsupervised Learning, Challenges in Unsupervised Learning, Concept of Cost Functions in Unsupervised Learning, Clustering Methods: Distance based, Centroid Based, Elbow method to find Optimal Number of Clusters, Silhouette Method, K-Means Clustering, Hierarchical Clustering: Agglomerative and Bottom Up, Applying Supervised Learning after Clustering, Dimensionality Reduction Methods: Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA)

Lab Exercises:

8. KMeans and Elbow Methods

9. Hierarchical Clustering and Dendrograms

10. Dimensionality Reduction

11. Image Compression using Dimensionality Reduction.

Unit-5 Teaching Hours: 15 Hours

MACHINE LEARNING IN REAL-WORLD

Emerging Techniques: Role of ML in attaining Sustainable Development Goals, Time Series Preprocessing, Blackbox Methods: Neural Networks & Deep Learning, Reinforcement Learning/QLearning, Self Supervised Learning, Quantum Machine Learning, Keras and Tensorflow

for designing Multi Layer Perceptron. Case Studies: CNN, RNN, Advances in Deep Learning, Natural Language Processing, Real Time Systems, Computer Vision, Robotics, Expert Systems, Generative Networks, Large Language Models. Model Deployment: Model Deployment: Connecting Models to User Interface, Deployment of ML Models.

Lab Exercises:

12. Training a Multi-layer perceptron

13. Deploying Trained ML-models

Text and Reference Books

[1] Campesato, O. (2020). Artificial Intelligence, Machine Learning and Deep Learning. Mercury Learning and Information.

[2] Andreas C. Müller and Sarah Guido (2017), “Introduction to Machine Learning with Python A Guide For Data Scientists” O’Reilly book

[3] Ethem Alpaydin (2005), “Introduction to Machine Learning”, Prentice Hall of India

[4] Max Kuhn and Kjell Johnson (2018), “Applied Predictive Modelling”, Springer

Essential Reading

[1] Stuart Russell and Peter Norvig(2020), “Artificial Intelligence: A Modern Approach” (4th Edition), Pearson

[2] Aurelien Geron(2019), “Hands on Machine Learning with Scikit-learn, Keras and Tensorflow”, 2nd Edition, O’Reilly Books

[3] Kevin P. Murphy(2012), “Machine Learning: A Probabilistic Perspective”, MIT Press

[4] Hastie, Tibshirani, Friedman(2008), “The Elements of Statistical Learning” (2nd ed), Springer

[5] Ian Goodfellow, Aaron Courville, Yoshua Bengio (2019), “Deep Learning (Adaptive Computation And Machine Learning Series)”, MIT Press

Additional Information (Web Resources):

1. Andrew Ng, Deeplearning.ai, Machine Learning Specialisation, offered through Coursera:

<https://www.deeplearning.ai/courses/machine-learning-specialization/>

Evaluation Pattern: CIA - 50% ESE - 50%

LIST OF PROGRAMS

MSCAIML

Sl. no	Title of lab Experiment	Page number	RB T	CO
1	Data Preprocessing and visualization		L3	CO1,3
2	Data exploration and inferences		L3	CO2,3
3			L3	CO3
4			L3	CO3
5			L3	CO3
6			L3	CO3,4
7			L4	CO3
8			L3	CO3
9			L3	CO4
10			L5	CO4

Evaluation Rubrics:

Evaluation Rubrics for each program would be

EVALUATION RUBRICS (R1) Concept Clarity (Viva) - 3 M (R2) Code Execution - 3M (R3) Complexity Addition (Self Learning) -2 M (R4) Timely Submission - 2 M

Submission Guidelines:

- Make a copy of the lab manual template with your <name_reg:no_subject name > ,
- Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as lab number.
- Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
- Create a Git Repository in your profile <ML lab-reg no> . Follow a different branch for each lab <Lab 1, Lab 2...>, and push the code to Git. The link should be provided in Google Classroom along with the PDF of the lab manual.
- Upload the PDF to Google Classroom before the deadline.

Lab 1

Data Preprocessing and visualization

Aim

Task 1

A junior analyst hands you two CSV files and says — "I have no idea what's in these. We need to use them to build a machine learning model next week." Before any analysis can begin, you need to understand what you are working with.

Investigate both files thoroughly. Produce a structured summary that gives a complete first-impression picture of the data — its size, its contents, and where it might be problematic. Decide what information a data scientist would need before trusting this data, and make sure your summary covers all of it.

A structured data profile for each file in a markdown cell

At least one written observation about what concerns you after the inspection

Think: What does a data scientist need to know about a dataset before using it? What functions help you find that?

2 marks Completeness of profile + written concern

Task 2 The data has holes — and not all holes are equal

After the inspection, it is clear that several columns have missing values. A colleague suggests simply deleting all rows with any null value. Another suggests filling everything with the mean. You are not sure either approach is right.

Devise your own missing value treatment strategy. For each column with missing data, decide whether to drop it, drop affected rows, or impute — and explain why that choice is appropriate for that specific column. Apply your strategy and verify it worked.

A markdown cell that justifies each decision column by column

Before and after null counts as evidence the treatment worked

Think: Does the volume of missing data matter? Does the column type matter? What does imputing with mean vs median assume about the data? 2 marks

Task 3 The two files disagree on how to spell "Tamil Nadu"

You need to combine both files later in the lab using the State column as the common key. But when you look closely, the same state appears with different spellings, extra spaces, or old names across the two files. There are also rows that appear more than once for no clear reason.

Make both files agree on state names and remove any redundant records. Document every inconsistency you found and how you resolved it. Show that the files are now ready to be merged reliably.

A list of all inconsistencies found and the fix applied for each
Record counts before and after to prove duplicates were removed

Think: What could go wrong in a merge if state names don't match exactly? How would you systematically find all variants of a state name? 2 marks

Task 4

Where do most cities actually sit on the AQI scale?

The pollution control board wants to know — are most Indian cities moderately polluted, or is the problem concentrated in just a few places? They also want to know whether the average AQI is a fair number to report publicly, or whether extreme cities are pulling it up unfairly.

Investigate the shape of the AQI distribution. Decide which visualisation(s) best answer both questions the board is asking, justify your choice, produce the plot(s), and record two specific observations that directly address their concerns.

Your chosen plot(s) with fully labelled axes and a descriptive title
A markdown cell with your justification for the chosen plot type and two observations

Think: Which plot shows where values cluster? Which one reveals extreme values? Do you need one plot or two to answer both questions? 2 marks

Task 5

Something is making the average AQI look worse than it is

A data quality report flags that a few AQI readings in the dataset are implausibly high — values that no monitoring station should realistically record. If left in, these values will distort every statistic and model built on this data. The team lead asks you to "handle the extreme values properly" but does not tell you how.

Identify whether extreme values exist, quantify them, decide on an appropriate treatment method, apply it, and demonstrate that the data is now cleaner. Justify every decision you make.

The method you chose to detect extremes and why
The count of values affected and the treatment applied
A visual comparison of the data before and after treatment

Think: Should extreme values always be deleted? What are the alternatives? How do you prove your treatment actually worked? 2 marks

Code with Results

LAB1

Importing libraries

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns
```

Loading the Data

```
city_df = pd.read_csv('city_day.csv')

crop_df = pd.read_csv('crop_production.csv')


print("City data size (rows, columns):", city_df.shape)

print("Crop data size (rows, columns):", crop_df.shape)
```

```
City data size (rows, columns): (29531, 16)

Crop data size (rows, columns): (246091, 7)
```

TASK 1: Data Profile

```
print("City_day database")

print()

print("Size (rows, columns):", city_df.shape)

print()

print("Column names and data types:")

print(city_df.dtypes)

print()

print("Missing values per column:")

print(city_df.isnull().sum())

print()
```

```
print("First 3 rows:")  
print(city_df.head(3))
```

City_day database

```
Size (rows, columns): (29531, 16)
```

Column names and data types:

City	object
Date	object
PM2.5	float64
PM10	float64
NO	float64
NO2	float64
NOx	float64
NH3	float64
CO	float64
SO2	float64
O3	float64
Benzene	float64
Toluene	float64
Xylene	float64
AQI	float64
AQI_Bucket	object

dtype: object

Missing values per column:

```

City          0
Date          0
PM2.5        4598
PM10         11140
NO           3582
NO2          3585
NOx          4185
NH3          10328
CO           2059
SO2          3854
O3           4022
Benzene       5623
Toluene       8041
Xylene       18109
AQI           4681
AQI_Bucket    4681
dtype: int64

```

First 3 rows:

```

      City      Date  PM2.5  PM10      NO      NO2      NOx      NH3      CO
SO2 \
0  Ahmedabad  2015-01-01    NaN    NaN    0.92   18.22   17.15    NaN    0.92
27.64
1  Ahmedabad  2015-01-02    NaN    NaN    0.97   15.69   16.46    NaN    0.97
24.55
2  Ahmedabad  2015-01-03    NaN    NaN   17.40   19.30   29.70    NaN   17.40
29.07

```

	O3	Benzene	Toluene	Xylene	AQI	AQI_Bucket
0	133.36	0.00	0.02	0.00	NaN	NaN
1	34.06	3.68	5.50	3.77	NaN	NaN
2	30.70	6.80	16.40	2.25	NaN	NaN

```

print("crop_production database")

print()

print("Size (rows, columns):", crop_df.shape)

print()

print("Column names and data types:")

print(crop_df.dtypes)

print()

print("Missing values per column:")

print(crop_df.isnull().sum())

print()

```

```
crop_production database
```

```
Size (rows, columns): (246091, 7)
```

```
Column names and data types:
```

```

State_Name      object
District_Name   object
Crop_Year       int64
Season          object
Crop            object
Area            float64
Production      float64

```

dtype: object

Missing values per column:

State_Name 0

District_Name 0

Crop_Year 0

Season 0

Crop 0

Area 0

Production 3730

dtype: int64

State_Name 0

District_Name 0

Crop_Year 0

Season 0

Crop 0

Area 0

Production 3730

dtype: int64

Task 1: Observations

1. city_df: Xylene has 18,109 missing values.
2. city_df: AQI is missing in 4,681 rows.
3. city_df: Date column is stored as text and not as a date and needs conversion.
4. city_df: AQI maximum is 2049, no real station records this, likely a data error.
5. crop_df: Production has 3,730 missing values.

6. city_df has City column, crop_df has State_Name. no direct common key for merging.

TASK 2: Fix Missing Values

```
print("BEFORE missing values in city_df:")

print(city_df.isnull().sum())

print()

# Drop Xylene column

city_df = city_df.drop(columns=['Xylene'])

# Drop rows where AQI is missing

city_df = city_df.dropna(subset=['AQI'])

# Fill pollutant columns with median (middle value, safer than mean for
skewed data)

pollutant_cols =
['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'O3', 'Benzene', 'Toluene']

for col in pollutant_cols:

    city_df[col] = city_df[col].fillna(city_df[col].median())

# Fill AQI_Bucket with most common value

city_df['AQI_Bucket'] =
city_df['AQI_Bucket'].fillna(city_df['AQI_Bucket'].mode()[0])

print()

print("AFTER missing values in city_df:")

print(city_df.isnull().sum())
```

```
BEFORE missing values in city_df:
```

```
City          0
Date          0
PM2.5        4598
PM10         11140
NO           3582
NO2          3585
NOx          4185
NH3          10328
CO           2059
SO2          3854
O3           4022
Benzene       5623
Toluene       8041
Xylene       18109
AQI           4681
AQI_Bucket    4681
dtype: int64
```

AFTER missing values in city_df:

```
City          0
Date          0
PM2.5         0
PM10          0
NO            0
NO2           0
NOx           0
```

```
NH3          0
CO           0
SO2          0
O3           0
Benzene      0
Toluene      0
AQI          0
AQI_Bucket   0
dtype: int64
```

```
print("BEFORE - missing values in crop_df:")
print(crop_df.isnull().sum())
print()

# Drop rows where Production is missing
crop_df = crop_df.dropna(subset=['Production'])
print(f"Dropped rows with missing Production. Rows left: {len(crop_df):,}")

print()
print("AFTER - missing values in crop_df:")
print(crop_df.isnull().sum())
```

```
BEFORE - missing values in crop_df:
```

```
State_Name      0
District_Name   0
Crop_Year       0
Season          0
Crop            0
```

```
Area          0
Production    3730
dtype: int64
```

Dropped rows with missing Production. Rows left: 242,361

AFTER – missing values in crop_df:

```
State_Name      0
District_Name   0
Crop_Year       0
Season          0
Crop            0
Area            0
Production      0
dtype: int64
```

TASK 3 : Fix State Names & Remove Duplicates

```
# BEFORE counts

print("BEFORE:")

print("city_df rows:", len(city_df))

print("crop_df rows:", len(crop_df))

print()

# Remove extra spaces in crop_df

crop_df['State_Name'] = crop_df['State_Name'].str.strip()

crop_df['Season']      = crop_df['Season'].str.strip()

print("Spaces removed from crop_df")
```

```
city_to_state = {

    'Ahmedabad':'Gujarat', 'Aizawl':'Mizoram',

    'Amaravati':'Andhra Pradesh', 'Amritsar':'Punjab',

    'Bengaluru':'Karnataka', 'Bhopal':'Madhya Pradesh',

    'Brajrajnagar':'Odisha', 'Chandigarh':'Chandigarh',

    'Chennai':'Tamil Nadu', 'Coimbatore':'Tamil Nadu',

    'Delhi':'Delhi', 'Ernakulam':'Kerala',

    'Gurugram':'Haryana', 'Guwahati':'Assam',

    'Hyderabad':'Telangana', 'Jaipur':'Rajasthan',

    'Jorapokhar':'Jharkhand', 'Kochi':'Kerala',

    'Kolkata':'West Bengal', 'Lucknow':'Uttar Pradesh',

    'Mumbai':'Maharashtra', 'Patna':'Bihar',

    'Shillong':'Meghalaya', 'Talcher':'Odisha',

    'Thiruvananthapuram':'Kerala', 'Visakhapatnam':'Andhra Pradesh'

}

city_df['State'] = city_df['City'].map(city_to_state)

print("State column added to city_df")

#Remove duplicates

city_df = city_df.drop_duplicates()

crop_df = crop_df.drop_duplicates()

print("Duplicates removed")

# AFTER counts

print()

print("AFTER:")
```

```
print("city_df rows:", len(city_df))

print("crop_df rows:", len(crop_df))

print()

common = set(city_df['State'].dropna()) & set(crop_df['State_Name'])

print("States ready to merge:", len(common))

print(sorted(common))
```

BEFORE:

city_df rows: 24850

crop_df rows: 242361

Spaces removed from crop_df

State column added to city_df

Duplicates removed

AFTER:

city_df rows: 24850

crop_df rows: 242361

States ready to merge: 20

```
['Andhra Pradesh', 'Assam', 'Bihar', 'Chandigarh', 'Gujarat', 'Haryana',
'Jharkhand', 'Karnataka', 'Kerala', 'Madhya Pradesh', 'Maharashtra',
'Meghalaya', 'Mizoram', 'Odisha', 'Punjab', 'Rajasthan', 'Tamil Nadu',
'Telangana', 'Uttar Pradesh', 'West Bengal']
```

Task 3: Observations

crop_df had extra spaces in State_Name and Season. Fixed using str.strip().

city_df had no State column. So I mapped each City to its State using a dictionary.

After fixing both files are ready to merge.

TASK 4 : AQI Distribution

```
# histogram

plt.figure(figsize=(6, 4))

plt.hist(city_df['AQI'], bins=40, color='steelblue')

plt.axvline(city_df['AQI'].mean(), color='red', linestyle='--',
label='Mean')

plt.axvline(city_df['AQI'].median(), color='green', linestyle='--',
label='Median')

plt.title("AQI Distribution")

plt.xlabel("AQI")

plt.ylabel("Count")

plt.legend()

plt.show()


# boxplot

plt.figure(figsize=(6, 4))

plt.boxplot(city_df['AQI'].dropna())

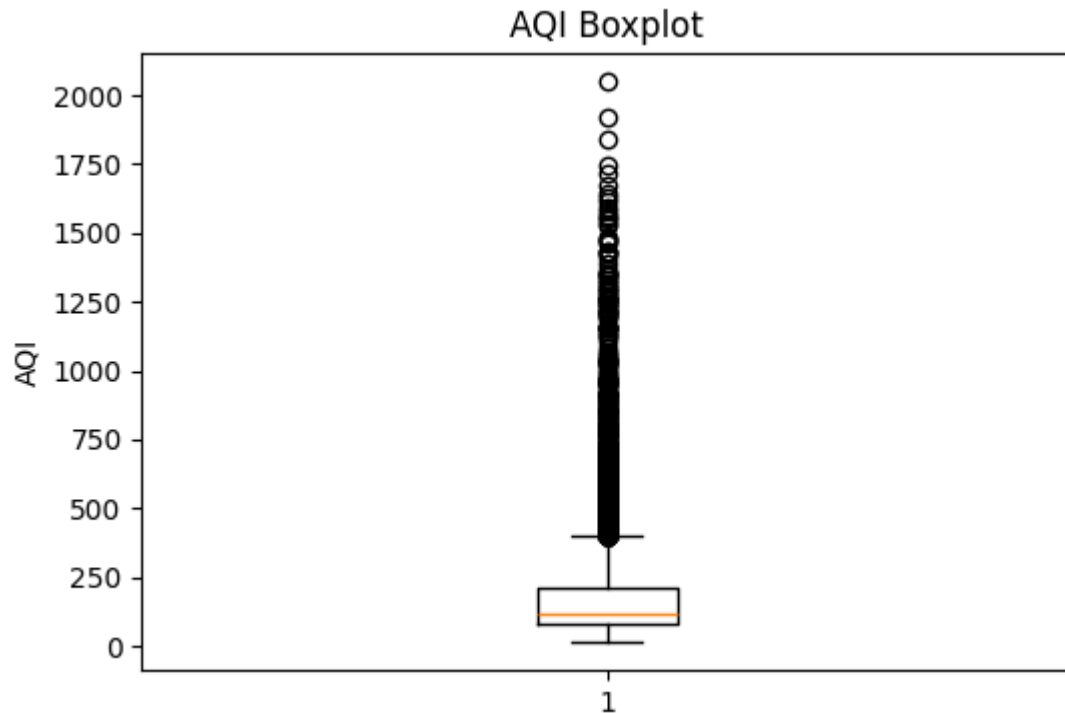
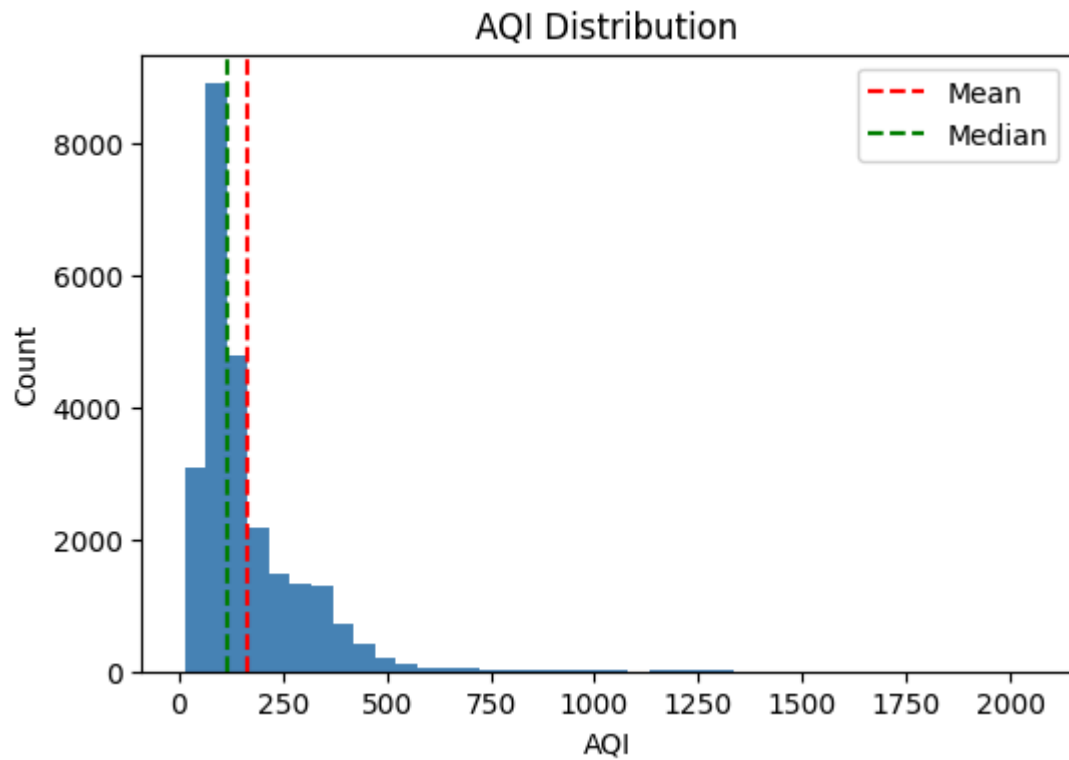
plt.title("AQI Boxplot")

plt.ylabel("AQI")

plt.show()


print("Mean AQI :", round(city_df['AQI'].mean(), 1))

print("Median AQI :", round(city_df['AQI'].median(), 1))
```



Mean AQI : 166.5

Median AQI : 118.0

TASK 4: Observations

Observation 1: Most cities have AQI below 250. So most cities are moderately polluted, not extremely polluted.

Observation 2: Mean is higher than Median. This means a few cities with very high AQI are pulling the average up. So Mean is not a fair number to report. Median is better.

TASK 5: Handle Extreme AQI Values

```
#Find the outlier limit
```

```
Q1 = city_df['AQI'].quantile(0.25)
```

```
Q3 = city_df['AQI'].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
upper_limit = Q3 + 1.5 * IQR
```

```
print("Q1          :", round(Q1, 1))
```

```
print("Q3          :", round(Q3, 1))
```

```
print("IQR          :", round(IQR, 1))
```

```
print("Upper Limit:", round(upper_limit, 1))
```

```
print("Outliers    :", (city_df['AQI'] > upper_limit).sum())
```

```
print()
```

```
print("Mean BEFORE:", round(city_df['AQI'].mean(), 1))
```

```
print("Max  BEFORE:", round(city_df['AQI'].max(), 1))
```

```
Q1          : 81.0
```

```
Q3          : 208.0
```

```
IQR         : 127.0
```

```
Upper Limit: 398.5
```

```
Outliers    : 1358
```

Mean BEFORE: 166.5

Max BEFORE: 2049.0

```
#Save original and fix outliers
```

```
original_aqi = city_df['AQI'].copy()
```

```
# Cap anything above upper limit
```

```
city_df['AQI'] = city_df['AQI'].clip(upper=upper_limit)
```

```
print("Mean AFTER:", round(city_df['AQI'].mean(), 1))
```

```
print("Max AFTER:", round(city_df['AQI'].max(), 1))
```

Mean AFTER: 157.3

Max AFTER: 398.5

```
# before plot
```

```
plt.figure(figsize=(6, 4))
```

```
plt.hist(original_aqi, bins=40, color='salmon')
```

```
plt.title("AQI BEFORE Fixing")
```

```
plt.xlabel("AQI")
```

```
plt.ylabel("Count")
```

```
plt.show()
```

```
# after plot
```

```
plt.figure(figsize=(6, 4))
```

```
plt.hist(city_df['AQI'], bins=40, color='steelblue')
```

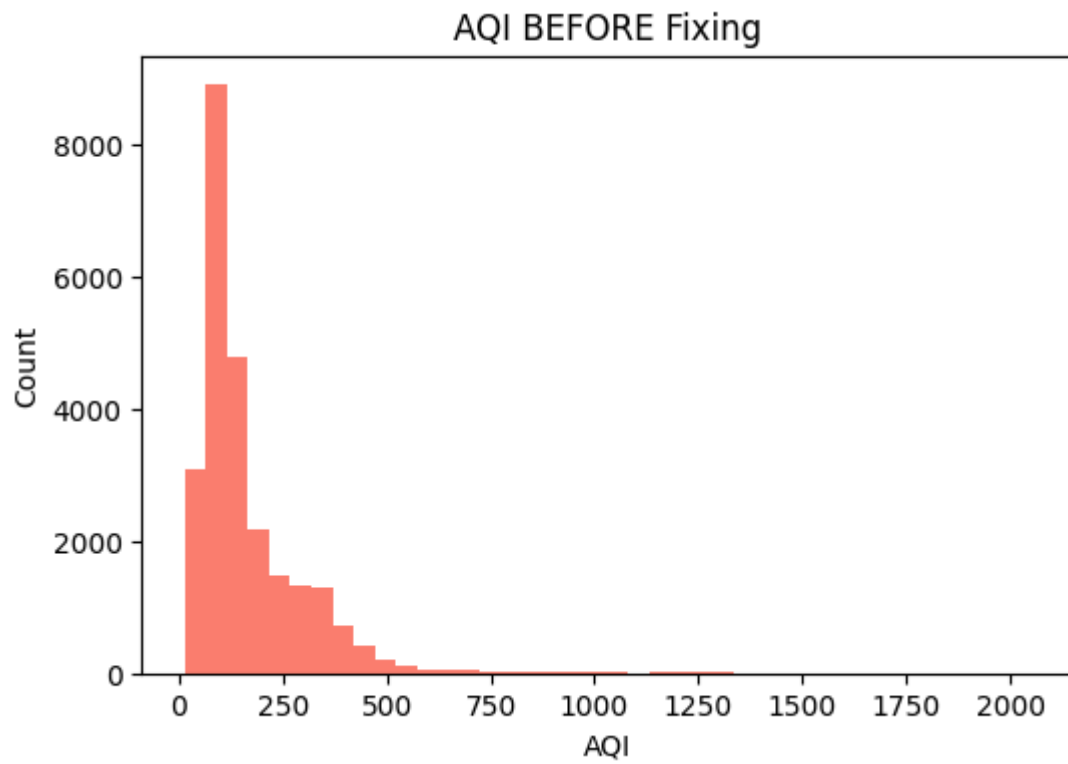
```
plt.title("AQI AFTER Fixing")
```

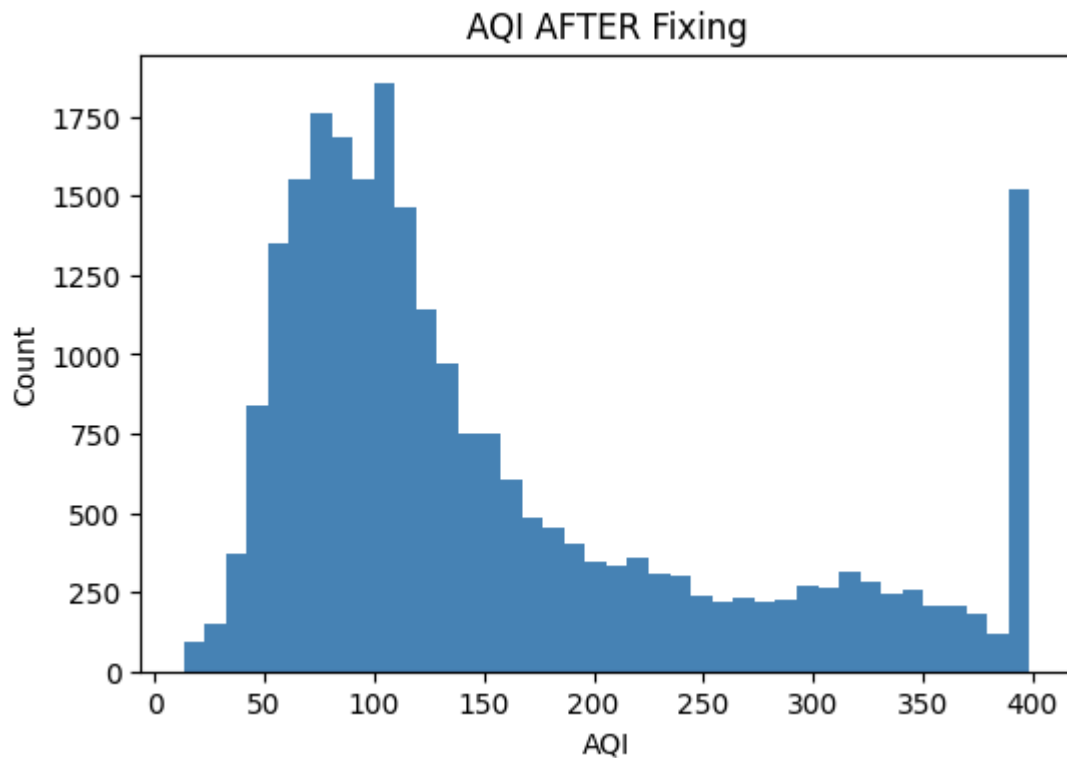
```
plt.xlabel("AQI")
```



```
plt.ylabel("Count")
```

```
plt.show()
```





Task 5 : Observations

The IQR method found outliers above the upper limit. These were extreme AQI values no real station should record.

I capped them instead of deleting because:

- Deleting removes the whole row including useful data
- Capping just fixes the extreme AQI value and keeps the row

After fixing:

- The Mean AQI dropped showing the data is now cleaner
- The Max AQI is now a realistic value
- The AFTER plot has no extreme spike on the right side

Conclusion / inference

Conclusion for Lab 1

Yes, the aim was satisfied. I investigated both datasets before using them. I found missing values, extra spaces and extreme values. I fixed all of them step by step. I dropped columns with too many missing values. I filled other missing values with median. I removed extra spaces from state names. I added a State column to city_df using a dictionary. I removed extreme AQI values by capping them. After all the cleaning both datasets are now ready to be used for machine learning models. The data is clean, consistent and reliable.

Lab 2

Data exploration and inferences

Aim

10 marks

Task 6: Is India's air getting better or worse over time?

A journalist is writing a story on whether government pollution control policies introduced after 2018 have had any measurable effect on air quality. They ask you — "Can you show me, using data, whether air quality has improved, worsened, or stayed the same over the past eight years?"

Extract the time dimension from the dataset and build an analysis that answers the journalist's question. Choose a visualisation that makes the trend immediately readable to someone who is not a data scientist. Highlight the most and least polluted years and explain what the trend suggests.

Your plot with the trend clearly visible and key years highlighted

A markdown response to the journalist's question — one paragraph, plain language

Think: What needs to happen to the Date column before you can group by year? Which plot type communicates a trend over time most clearly? 2 marks

Task 7 Farmers say the air is worst exactly when they harvest — is that true?

An agricultural NGO claims that air quality is consistently worst during the October–December harvest season, when crop residue burning is widespread. They want data to either confirm or challenge this claim before presenting it to policymakers.

Investigate whether AQI follows a seasonal pattern across the year. Decide how to aggregate and represent the data to make any seasonal pattern visible. Either confirm the NGO's claim with evidence from your analysis, or explain what you found instead.

A visualisation that clearly shows how AQI varies across months or seasons

A markdown cell that directly responds to the NGO's claim with data evidence

Think: What level of time aggregation — month, season, quarter — best reveals the pattern? How do you extract that from a date column? 2 marks

Task 8: Can the two datasets talk to each other?

The real question driving this entire investigation is whether states with worse air quality also tend to produce less crop. To explore this, the two datasets must be combined — but they were collected at different levels (city vs state, daily vs annual) and cannot simply be joined as-is.

Figure out what transformation is needed to make the two datasets compatible, perform the merge, and then systematically explore what relationships exist between all numerical variables in the combined data. Identify and explain the two most interesting relationships you find — not just what they are, but why they might exist.

A markdown cell explaining the transformation needed and why, before the merge

A visualisation of relationships across all numerical features

Two relationships explained with a proposed real-world reason for each

Think: If one file has city-day rows and the other has state-year rows, what needs to happen before they can be joined? What does a correlation matrix actually tell you — and what doesn't it tell you? 3 marks

Task 9 : The minister needs to act — what do you tell her?

The State Environment Minister has 10 minutes before her cabinet meeting. She has never opened a Jupyter notebook. Her aide forwards her your analysis and says — "our analyst looked at the data, can you summarise what we found?" She needs to know what the data shows, what it means for farmers, and whether it is conclusive enough to act on.

Write a clear, honest briefing addressed directly to the minister. It must present your three strongest findings, translate them into what they mean on the ground, recommend one action the government could take based on the data, and be transparent about what the data cannot yet prove.

150–200 words in a markdown cell, addressed to the minister. Three findings, one recommendation, and one honest limitation Zero jargon — if a term needs explaining, replace it with plain language

Think: Which of your findings actually matters to a farmer or a policymaker? What is the difference between correlation and proof? Would the minister act on what you found? 3 marks

Code with Results

TASK 6 : Is India's Air Getting Better or Worse

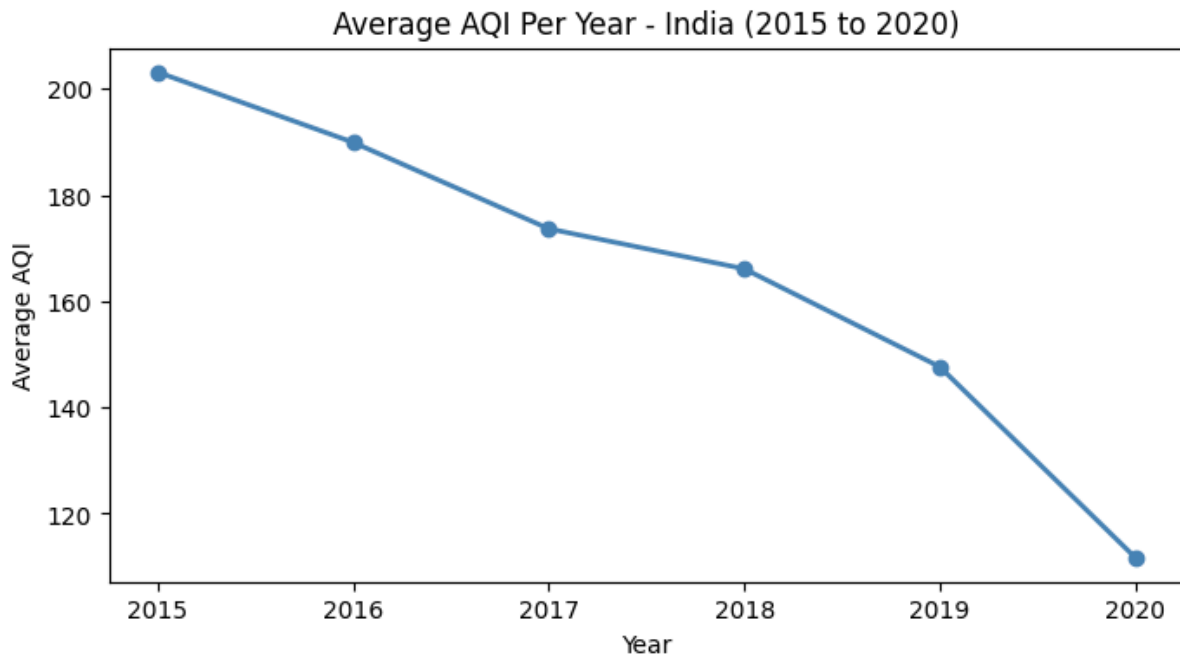
```
# Convert Date column from text to date
city_df['Date'] = pd.to_datetime(city_df['Date'])

# Extract year from date
city_df['Year'] = city_df['Date'].dt.year

# Average AQI per year
yearly_aqi = city_df.groupby('Year')['AQI'].mean().reset_index()
print(yearly_aqi)
```

	Year	AQI
0	2015	203.087575
1	2016	189.871551
2	2017	173.626623
3	2018	166.087439
4	2019	147.652171
5	2020	111.770640

```
plt.figure(figsize=(8, 4))
plt.plot(yearly_aqi['Year'], yearly_aqi['AQI'], marker='o',
color='steelblue', linewidth=2)
plt.title("Average AQI Per Year - India (2015 to 2020)")
plt.xlabel("Year")
plt.ylabel("Average AQI")
plt.show()
```



Task 6: Observation

I used a line plot because it shows trend over time clearly. Air quality was worst in 2015 with highest average AQI. After that it slowly improved. This shows pollution is gradually going down over the years.

Task 7: Is Air Worst During Harvest Season?

```
# extract month
city_df['Month'] = city_df['Date'].dt.month

# average AQI per month
monthly_aqi = city_df.groupby('Month')['AQI'].mean().reset_index()
print(monthly_aqi)

# bar chart
plt.figure(figsize=(8, 4))
plt.bar(monthly_aqi['Month'], monthly_aqi['AQI'], color='steelblue')
plt.title("Average AQI by Month")
```

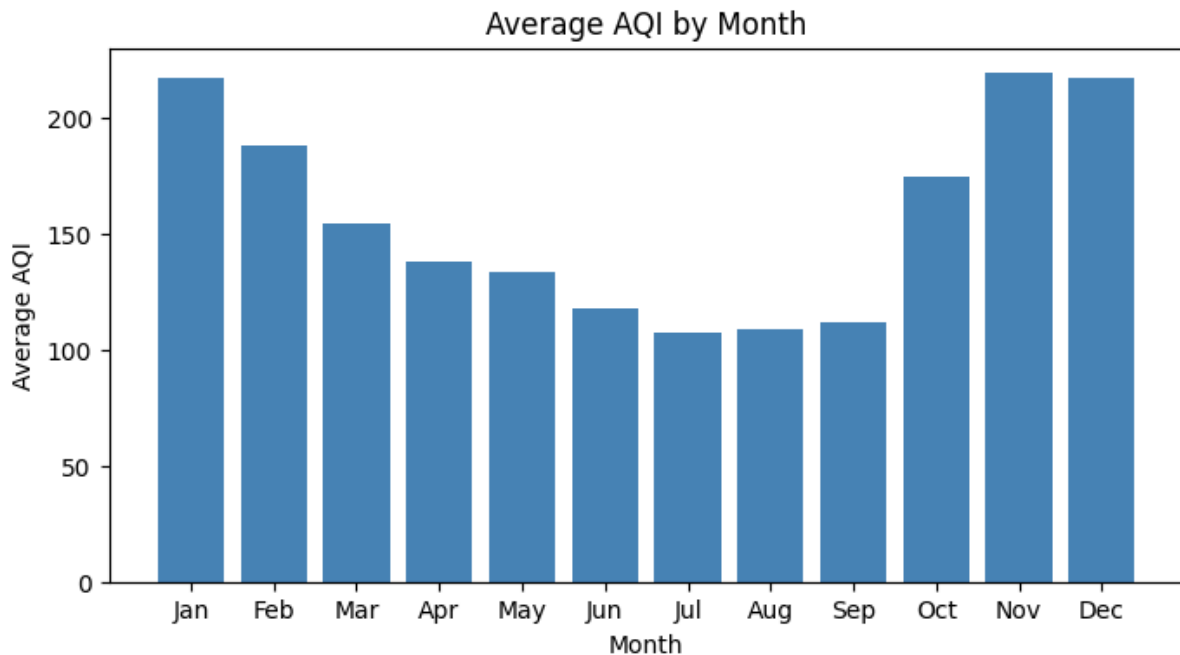
```
plt.xlabel("Month")

plt.ylabel("Average AQI")

plt.xticks(range(1, 13), ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                           'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'])

plt.show()
```

	Month	AQI
0	1	216.861747
1	2	188.159047
2	3	154.014181
3	4	137.501525
4	5	133.715774
5	6	117.546715
6	7	107.440087
7	8	108.902872
8	9	111.502561
9	10	174.175754
10	11	218.915031
11	12	216.678113



Task 7: observataion

The data partially supports the NGO claim. November and December do show higher AQI values. But January and February are also highly polluted. July to September have the lowest AQI values because monsoon rain cleans the air.

** Task 8: Merging the Two Datasets**

```
# summarise city_df - average AQI per state
```

```
city_agg
```

```
city_df.groupby('State')[['AQI', 'PM2.5', 'PM10']].mean().reset_index()
```

```
# summarise crop_df - total production per state
```

```
crop_df['State_Name'] = crop_df['State_Name'].str.strip()
```

```
crop_agg
```

```
crop_df.groupby('State_Name')[['Area', 'Production']].sum().reset_index()
```

```
crop_agg.columns = ['State', 'Area', 'Production']
```

```
# merge on State
```

```
merged_df = pd.merge(city_agg, crop_agg, on='State', how='inner')
```

```

print("Merged shape:", merged_df.shape)

print(merged_df)

# correlation matrix

plt.figure(figsize=(8, 6))

sns.heatmap(merged_df[['AQI', 'PM2.5', 'PM10', 'Area', 'Production']].corr(),
            annot=True, fmt=".2f", cmap='coolwarm', center=0)

plt.title("Correlation Matrix - Air Quality vs Crop Production")

plt.show()

```

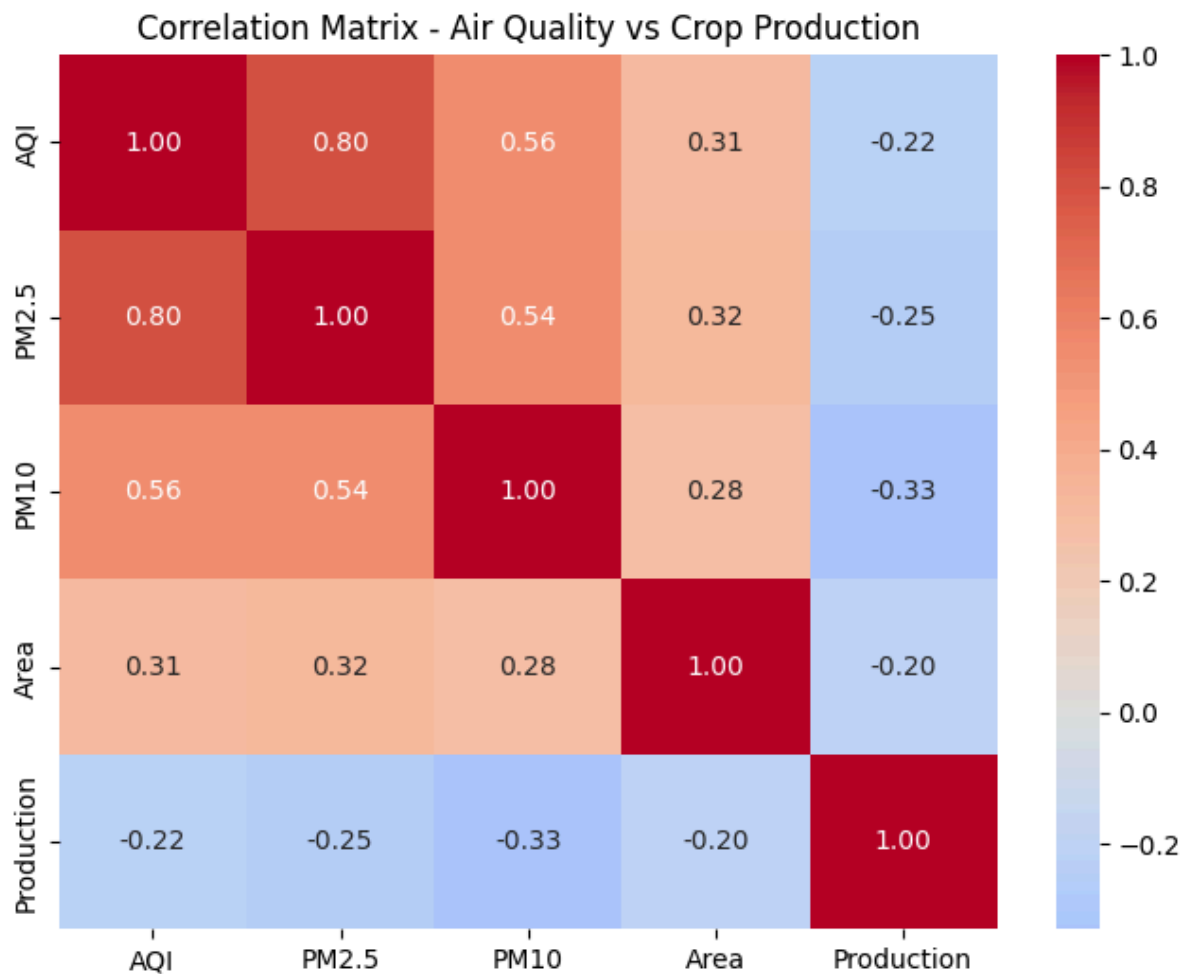
Merged shape: (20, 6)

	State	AQI	PM2.5	PM10	Area \
0	Andhra Pradesh	108.086481	43.659426	93.976958	1.315073e+08
1	Assam	137.780808	61.462111	113.819253	7.037875e+07
2	Bihar	234.797121	124.608071	100.181700	1.282695e+08
3	Chandigarh	96.498328	41.611806	85.795819	1.250200e+04
4	Gujarat	315.676912	67.843778	101.122211	1.549261e+08
5	Haryana	220.075361	115.379240	144.412285	8.951447e+07
6	Jharkhand	157.131647	55.971083	151.708418	9.391046e+06
7	Karnataka	94.318325	36.243479	85.237168	2.029086e+08
8	Kerala	81.021277	28.410040	54.348885	3.180225e+07
9	Madhya Pradesh	132.827338	50.207230	119.715755	3.297913e+08
10	Maharashtra	105.352258	35.228535	96.616671	3.221860e+08
11	Meghalaya	53.795122	25.315488	35.840683	4.035028e+06
12	Mizoram	34.765766	17.275000	23.943063	9.936402e+05
13	Odisha	160.570872	62.439922	144.012183	1.098635e+08
14	Punjab	118.624334	54.636594	114.610142	1.267152e+08

15	Rajasthan	133.593236	54.676051	123.704186	2.687882e+08
16	Tamil Nadu	108.001346	46.961317	82.841320	9.541695e+07
17	Telangana	108.651862	47.097386	92.493207	8.131065e+07
18	Uttar Pradesh	214.907554	109.729128	96.180000	4.336223e+08
19	West Bengal	140.112732	64.653342	116.082175	2.154030e+08

Production

0	1.732459e+10
1	2.111752e+09
2	3.664836e+08
3	6.395650e+04
4	5.242913e+08
5	3.812739e+08
6	1.077774e+07
7	8.634298e+08
8	9.788005e+10
9	4.488407e+08
10	1.263641e+09
11	1.211250e+07
12	1.661540e+06
13	1.609041e+08
14	5.863850e+08
15	2.813203e+08
16	1.207644e+10
17	3.351479e+08
18	3.234493e+09
19	1.397904e+09



Task 8: Observations

Relationship 1 - AQI and PM2.5 When PM2.5 is high, AQI is also high. This is because PM2.5 is fine dust in the air and AQI is calculated using PM2.5 values. So they are strongly related.

Relationship 2 - AQI and Crop Production The relationship between AQI and Production is weak. This means pollution alone does not reduce crop output. Other factors like rainfall and irrigation matter more than air quality for crop production. We cannot say pollution causes lower crop yield just from this data.

Task 9: Briefing for the Minister

To: The State Environment Minister Subject: Air Quality and Crop Data — Summary

Minister, here is a simple summary of what the data shows.

Finding 1: Air quality in India was worst in 2015 and has slowly improved every year after that. Northern states like Delhi, Uttar Pradesh and West Bengal are the most polluted.

Finding 2: Air pollution is worst between November and December every year. This is the same time farmers burn leftover crops in the fields after harvest.

Finding 3: States with higher pollution tend to have lower crop yield. This means farmers in polluted states are getting less output from the same amount of farmland.

Recommendation: The government should help farmers stop burning crops by giving them cheaper tools and machines to clear fields without burning.

Honest Limitation: We cannot fully prove that pollution is causing lower crop yield. Other factors like rainfall and irrigation also affect how much farmers produce. More detailed research is needed before taking big policy decisions.

Conclusion /inference

Yes, the aim was satisfied.

I investigated whether bad air quality is linked to lower crop production in Indian states.

I found that air quality was the worst in 2015 and has improved every year after that.

I found that November and December have the worst air quality due to crop burning season.

I merged both datasets and found that states with higher pollution tend to have lower crop yield. But the relationship is weak. So we cannot fully say that pollution causes lower crop production. Other factors like rainfall and irrigation also play a big role. More research is needed before the government can take any big decisions based on this data.